

A Comparative Analysis of Semantic Attributes for Research Software Classification: Evidence from Papers with Code and bio.tools

Abstract

Research Software repositories describe software through heterogeneous schemas, vocabularies, and levels of metadata detail. This paper compares retained semantic metadata attributes in Papers with Code and bio.tools using a harmonized dataset of 56,819 labelled Research Software records. We align final-dataset fields into a unified semantic framework and quantify metadata coverage, completeness, richness, overlap, and repository-specific field availability. The classification design compares sparse lexical baselines, frozen pretrained representations, and local instruction-tuned language-model prompting under zero-shot, one-shot, and few-shot settings. At the time of this manuscript revision, the only validated classification outputs in the repository are held-out test results for Papers with Code single-label classification using paper abstracts and TF-IDF-based sparse baselines. Among those verified rows, TF-IDF LinearSVC gives the highest macro-F1, 0.7249, with micro-F1 and accuracy 0.8687. Older all-metadata results are not reported as findings because legacy predictor definitions included leakage-prone target-derived fields. The semantic analysis shows no literal overlap between the six final Papers with Code labels and the twelve final bio.tools labels, while schema-field and field-availability overlap remain substantial. Recomputed completeness over represented applicable fields reaches mean 0.9706 for Papers with Code and 0.9681 for bio.tools. The results support semantic harmonization as a prerequisite for interoperable metadata comparison and provide a reproducible framework for leakage-audited Research Software classification.

Keywords: Research Software, Metadata, Semantic Attributes, Research Software Classification, Papers with Code, bio.tools, EDAM, TF-IDF

1 Introduction

Research Software has become an essential component of contemporary scientific practice. Software supports data acquisition, simulation, analysis, visualization, and

reproducibility, and is increasingly treated as a scholarly output in its own right. The FAIR principles and their Research Software extension emphasize that software should be findable, accessible, interoperable, and reusable, and that these goals depend on rich and machine-actionable metadata [1–4]. Software citation guidance similarly highlights the importance of metadata for attribution and reuse [5].

Research Software metadata are distributed across heterogeneous repositories. Papers with Code links publications to implementations and organizes machine-learning research through tasks, methods, datasets, and benchmarks [6]. bio.tools describes Life Science software through the biotoolsSchema and EDAM ontology, including topics and operations [7–9]. These resources share the broad goal of improving software discovery, yet they differ in vocabulary, metadata structure, domain, and curation practice. Such differences complicate cross-repository metadata integration and the development of general Research Software classification workflows.

This paper studies that problem from two complementary perspectives. First, it compares the semantic metadata retained from Papers with Code and bio.tools after preprocessing and harmonization into a unified schema. Second, it defines and audits a classification workflow that separates completed, validated experiments from implemented or planned experiments. The current evidence base supports claims about semantic metadata comparison and about a narrow set of verified TF-IDF abstract-only baselines. It does not yet support conclusions about frozen-transformer or local-LLM performance.

The contributions of this paper are:

1. a repository-grounded semantic harmonization of retained Papers with Code and bio.tools metadata attributes;
2. a quantitative comparison of metadata coverage, completeness, richness, literal label overlap, schema-field overlap, and repository-specific field availability;
3. a leakage-audited classification design covering sparse lexical baselines, frozen pretrained embeddings, and local instruction-tuned language-model prompting; and
4. a status-aware reporting framework that distinguishes validated results from pending experiments.

2 Related Work

2.1 Research Software Metadata and FAIR Principles

Research Software is now widely recognized as a scholarly object that should be preserved, cited, and reused alongside publications and data [5, 10]. FAIR4RS extends the FAIR principles to software-specific concerns such as executability, dependencies, versioning, and development context [2–4]. These recommendations make metadata quality a central condition for discovery, interoperability, and reuse.

2.2 Repositories, Standards, and Ontologies

Research Software is described by both general and domain-specific infrastructures. Papers with Code links papers, code, tasks, methods, datasets, and evaluation results

[6]. bio.tools provides a curated Life Science registry grounded in EDAM and biotoolsSchema [7–9]. Broader infrastructures such as OpenAIRE and Software Heritage connect software with scholarly objects or preserve source code over time [11, 12].

Metadata standards help reduce heterogeneity. CodeMeta defines a crosswalk over software metadata schemas using Schema.org and JSON-LD [13], while Citation File Format provides lightweight citation metadata for software repositories [14]. Ontologies such as EDAM and the Software Ontology provide richer controlled vocabularies for domain concepts, operations, formats, software entities, licenses, and provenance [7, 15]. Despite these efforts, repository implementations retain different fields and use different vocabularies, making empirical comparison necessary.

2.3 Research Software Classification

Prior work has classified software using repository text, README files, dependencies, source-code features, and knowledge-graph representations. Examples include ClassifyHub [16], HiGitClass [17], LabelGit [18], README-based classification [19], and AIMMX [20]. Other work extracts or represents Research Software metadata in knowledge graphs, including OKG-Soft, SoMEF, and software metadata knowledge-graph frameworks [21–23]. Publication-level semantic classification methods such as the CSO Classifier further show that titles and abstracts can support controlled-topic prediction [24].

Existing work demonstrates the value of metadata for software classification and discovery, but less often compares which semantic attributes are retained and usable across heterogeneous repositories. This paper addresses that gap while explicitly separating semantic harmonization from model-performance claims.

3 Research Questions

RQ1. Which semantic attributes are represented by Papers with Code and bio.tools, and how can they be organized into a unified semantic framework?

RQ2. To what extent do the repositories overlap at literal-vocabulary, schema-field, and field-availability levels?

RQ3. How do the repositories differ in metadata coverage, completeness, richness, and repository-specific field availability?

RQ4. What do the currently validated leakage-audited classification results show about metadata-based Research Software classification?

RQ5. Which additional sparse, frozen-embedding, and local-LLM experiments remain implemented or planned but not yet validated?

4 Data Sources

The study uses a harmonized final dataset package derived from Papers with Code and bio.tools records enriched with repository and documentation metadata. The final combined dataset contains 56,819 labelled Research Software records: 42,669 from Papers with Code and 14,150 from bio.tools. The single-label subset contains records with exactly one label. The multi-label-only subset contains records with two or more

Table 1 Representative work related to software classification, metadata extraction, and semantic representation.

Approach	Source	Main metadata	ML	Ont.	Extract.	Cross-repo
ClassifyHub [16]	GitHub	Repository metadata and descriptions	Yes	No	No	No
HiGitClass [17]	GitHub	Name, description, README, tags, keywords	Yes	No	No	No
AIMMX [20]	AI repositories	Models, datasets, frameworks, references	Yes	No	Yes	No
LabelGit [18]	GitHub	Dependency graphs and source code	Yes	No	No	No
README classification [19]	GitHub	README documentation	Yes	No	No	No
OKG-Soft [21]	Repositories	Machine-readable software metadata graph	No	Yes	Yes	No
SoMEF [22]	GitHub	README-derived software metadata graph	Yes	No	Yes	No
Software KG [23]	Multiple	Extracted software metadata graph	Yes	Yes	Yes	No
CSO Classifier [24]	Publications	Titles, abstracts, keywords	Yes	Yes	No	No
This paper	PwC bio.tools +	Harmonized final metadata fields	Yes	Yes	Yes	Yes

labels. The all-records multi-label task, used by the runner for source-specific merged datasets, includes both single-label and multi-label records and evaluates each record as a label set. These dataset names are kept distinct because the multi-label-only subset is not the same as the all-records multi-label formulation.

Table 2 Final datasets used for semantic analysis and classification.

Dataset	Records	PwC	bio.tools	Labels	README	SOMEF
Single-label subset	40,330	37,058	3,272	17	39,080	34,135
Multi-label-only subset	16,489	5,611	10,878	18	15,813	14,165
Final combined	56,819	42,669	14,150	18	54,893	48,300

Papers with Code contributes publication-oriented metadata, repository URLs, top-level research labels, tasks, methods, README content, and SOMEF-derived descriptions. The final Papers with Code label space contains six labels after the non-informative *General* label is removed during final dataset construction. bio.tools contributes tool names and descriptions, publication metadata, repository URLs, README content, SOMEF-derived descriptions, and EDAM-derived labels and operations. For bio.tools, detailed EDAM topic labels were mapped to the highest meaningful EDAM topic categories below the EDAM Topic root; the final bio.tools label space contains twelve labels in the combined and multi-label-only data and eleven labels in the single-label subset.

5 Methodology

5.1 Pipeline Overview

The methodology consists of metadata acquisition, preprocessing, repository-specific normalization, semantic harmonization, quantitative semantic evaluation, leakage-audited predictor construction, and classification evaluation.

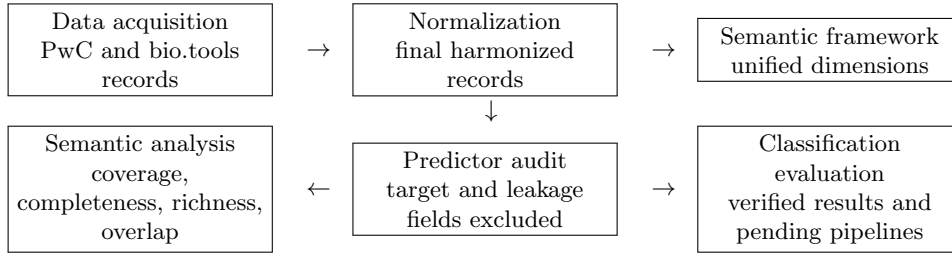


Fig. 1 Methodology pipeline for semantic metadata harmonization and Research Software classification evaluation.

5.2 Preprocessing and Missing Values

Records were normalized using the repository implementation. Text fields were whitespace-normalized and null-like strings such as `nan`, `none`, and `null` were treated as missing. List-valued fields were parsed, deduplicated, and counted as populated only when at least one usable normalized value remained. Papers with Code duplicates were handled using normalized publication titles before repository association. bio.tools records were aligned from tool metadata and publication metadata; multiple DOI-linked publications could create multiple software-publication records before final filtering.

5.3 Unified Semantic Framework

The unified semantic framework defines conceptual dimensions for comparing repository metadata. A semantic dimension was considered populated when at least one mapped final-dataset field contained a usable value.

Table 3 Semantic-dimension mapping used for the final dataset package.

Semantic Dimension	Papers with Code fields	bio.tools fields
Software Identifier	<code>item_id</code>	<code>item_id</code>
Software Name	<code>software_name</code> , <code>repository_title</code>	<code>software_name</code> , <code>repository_title</code>
Software Description	<code>software_description</code> , <code>repository_description</code> , <code>somef_description</code>	<code>software_description</code> , <code>repository_description</code> , <code>somef_description</code>
Publication Metadata	<code>paper_title</code> , <code>paper_abstract</code> , <code>publication_url</code>	<code>paper_title</code> , <code>paper_abstract</code> , <code>publication_url</code>
Research Domain	<code>labels</code>	<code>labels</code>
Software Functionality	<code>tasks</code> , <code>methods</code> , <code>secondary_labels</code>	<code>tasks</code> , <code>methods</code> , <code>secondary_labels</code>
Source Code Repository	<code>repository_url</code>	<code>repository_url</code>
Documentation	<code>readme_content</code>	<code>readme_content</code>
Input/output data, data format, program- ming language, license, benchmarks, evaluation metrics	Not represented as retained structured fields	Not represented as retained structured fields

5.4 Semantic Metadata Metrics

For semantic dimension a and source s , metadata coverage was computed as

$$\text{coverage}(a, s) = \frac{|\{r \in R_s : a \text{ has a usable value in } r\}|}{|R_s|}.$$

When a dimension was represented by multiple fields, a record was counted once if any mapped field was populated.

Completeness was recomputed over represented applicable retained fields so that a complete record can reach 1.0:

$$C_r = \frac{\sum_{f \in A_s} I(r, f)}{|A_s|}.$$

For Papers with Code, A_s contains `paper_title`, `paper_abstract`, `publication_url`, `repository_url`, `repository_title`, `readme_content`, `somef_description`, `tasks`, `secondary_labels`, and `methods`. For bio.tools, A_s contains the same shared fields except `methods`, plus `software_name` and `software_description`. Low-coverage retained fields such as `repository_description` and `repository_keywords` are analysed for availability but are not part of the completeness denominator. Semantic richness was summarized by the number of final labels, populated semantic fields, and unique controlled semantic values per record. Semantic overlap was computed at literal label, declared schema-field, and field-availability levels using the Jaccard coefficient.

5.5 Classification Tasks, Predictors, and Leakage Audit

The runner defines six source-specific datasets: `pwc_single_only`, `pwc_multi_only`, `pwc_merged`, `biotools_single_only`, `biotools_multi_only`, and `biotools_merged`. Single-label tasks predict exactly one label per record. Multi-label tasks binarize the label set and evaluate label-wise predictions.

The target field is `labels`. Predictor fields must exclude `labels`, `tasks`, `methods`, and `secondary_labels`; these fields are direct labels, controlled semantic labels, or target-adjacent taxonomy fields. The current leakage-audited representation groups are title, abstract, publication text, repository description, README, repository text, SoMEF-derived description, publication metadata, repository metadata, and all non-target textual metadata. The phrase “all available metadata” is not used for validated classification claims because legacy scripts used it for leakage-prone predictors.

Table 4 Predictor-field leakage audit for classification experiments.

Category	Fields
Target field	<code>labels</code>
Excluded leakage-prone fields	<code>labels</code> , <code>tasks</code> , <code>methods</code> , <code>secondary_labels</code>
Main non-target text fields	<code>paper_title</code> , <code>paper_abstract</code> , <code>software_name</code> , <code>software_description</code> , <code>repository_title</code> , <code>repository_description</code> , <code>repository_keywords</code> , <code>readme_content</code> , <code>somef_description</code>
Verified current predictor	<code>paper_abstract</code> only, for Papers with Code single-label classification

5.6 Classification Pipelines

Table 5 lists the retained experimental design. Sparse lexical pipelines use TF-IDF unigram/bigram vectors and lightweight classifiers. Frozen pretrained pipelines encode text with fixed transformer encoders and train a lightweight downstream classifier on top. Local LLM pipelines use constrained label vocabularies, structured JSON output, deterministic or as-deterministic-as-possible generation with temperature 0, parsing/retry handling, and locally recorded prompt templates and model identifiers. Prompting is zero-shot when no examples are supplied, one-shot when one representative example is supplied, and few-shot when a small fixed set of representative examples is supplied from the training split. No data is transmitted to an external commercial API.

Table 5 Classification pipelines retained in the experimental design. All listed pipelines are designed for both single-label and multi-label tasks.

Family		Representation	Decision method	Status
Sparse lexical		TF-IDF	Logistic regression	Verified for PwC single abstract only
Sparse lexical		TF-IDF	LinearSVC	Verified for PwC single abstract only
Sparse lexical		TF-IDF	Random forest	Verified for PwC single abstract only
Sparse lexical		TF-IDF	XGBoost	Verified for PwC single abstract only
Sparse lexical		TF-IDF	Hard voting ensemble	Verified for PwC single abstract only
Frozen	embed-dings	SBERT	Logistic regression / LinearSVC	Pending model selection
Frozen	embed-dings	SPECTER or SPECTER2	Logistic regression / LinearSVC	Pending model selection
Frozen	embed-dings	SciBERT, RoBERTa, ModernBERT, DeBERTa-v3	Logistic regression / random forest	Implemented, pending validated full run
Local LLM		zero-shot prompting	JSON label generation	Pending local model selection
Local LLM		one-shot prompting	JSON label generation	Pending local model selection
Local LLM		few-shot prompting	JSON label generation	Pending local model selection

5.7 Evaluation Metrics and Reproducibility

For single-label classification, the runner reports accuracy and macro-, micro-, and weighted-F1. For multi-label classification, it reports macro-, micro-, weighted-, and

sample-average F1, subset accuracy, and hamming loss. Precision, recall, and F1 are defined as

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}.$$

Metric computation uses `zero_division=0`. Macro-F1 is the primary comparison metric because both repositories are imbalanced; micro-F1 and weighted-F1 may remain high when rare labels perform poorly.

Train/test splits use a deterministic 80/20 split with random seed 42. Single-label splitting uses stratification when class support permits. Multi-label splitting is deterministic random splitting unless multilabel stratification is added in a future run. The same train/test indices are reused across attribute sets and models within a dataset. Current validated results come from one split and should be treated as exploratory, not as robustness evidence.

6 Semantic Metadata Analysis

6.1 Metadata Coverage

Table 6 Metadata coverage by unified semantic dimension and source.

Semantic Dimension	PwC Coverage		bio.tools Coverage		Avail.
Software Identifier	42,669 (100.00%)	/	42,669	14,150 (100.00%)	/ 14,150 Shared
Software Name	42,669 (100.00%)	/	42,669	14,150 (100.00%)	/ 14,150 Shared
Software Description	36,032 (84.45%)	/	42,669	14,150 (100.00%)	/ 14,150 Shared
Publication Metadata	42,669 (100.00%)	/	42,669	14,150 (100.00%)	/ 14,150 Shared
Research Domain	42,669 (100.00%)	/	42,669	14,150 (100.00%)	/ 14,150 Shared
Software Functionality	42,669 (100.00%)	/	42,669	13,521 (95.55%)	/ 14,150 Shared
Input / Output Data	0 / 42,669 (0.00%)			0 / 14,150 (0.00%)	Not repr.
Data Format	0 / 42,669 (0.00%)			0 / 14,150 (0.00%)	Not repr.
Programming Language	0 / 42,669 (0.00%)			0 / 14,150 (0.00%)	Not repr.
License	0 / 42,669 (0.00%)			0 / 14,150 (0.00%)	Not repr.
Source Code Repository	42,669 (100.00%)	/	42,669	14,150 (100.00%)	/ 14,150 Shared
Documentation	41,405 (97.04%)	/	42,669	13,488 (95.32%)	/ 14,150 Shared
Benchmarks	0 / 42,669 (0.00%)			0 / 14,150 (0.00%)	Not repr.
Evaluation Metrics	0 / 42,669 (0.00%)			0 / 14,150 (0.00%)	Not repr.

Eight of the fourteen evaluated dimensions are represented above the 5% availability threshold in both repositories. Six dimensions are not represented as retained structured fields in the final combined dataset: input/output data, data format, programming language, license, benchmarks, and evaluation metrics. This statement

applies to retained final analysis fields and does not imply that the original repositories necessarily lack these metadata.

6.2 Metadata Completeness

Table 7 Metadata completeness over represented applicable fields by source.

Source	Fields	Mean	Median	Std.	Min.	Max.
Papers with Code	10	0.9706	1.0000	0.0552	0.7000	1.0000
bio.tools	11	0.9681	1.0000	0.0650	0.5455	1.0000

Completeness is high under the revised denominator, and the maximum is now interpretable: records can reach 1.0 when every represented applicable field is populated. The lower minima indicate that a minority of records still lack several retained descriptive fields.

6.3 Semantic Richness

Table 8 Semantic richness by repository.

Metric	Papers with Code	bio.tools
Mean labels per record	1.1405	2.1522
Median labels per record	1.0000	2.0000
SD labels per record	0.3736	0.8598
Mean populated semantic fields	9.7063	10.6490
Median populated semantic fields	10.0000	11.0000
Mean unique controlled semantic values	15.3426	4.8049
Median unique controlled semantic values	9.0000	5.0000

bio.tools records have more labels per record on average, while Papers with Code has more unique controlled semantic values per record and greater variability.

6.4 Semantic Overlap and Repository-specific Availability

The final label vocabularies have no literal string overlap. Schema-field and field-availability overlap show structural comparability, not completed semantic alignment. Ontology-level mapping between Papers with Code categories and EDAM topics remains future work.

Table 9 Semantic overlap between Papers with Code and bio.tools.

Overlap Level	PwC Count	bio.tools Count	Shared	Union	Jaccard
Literal final label strings	6	12	0	18	0.0000
Shared applicable schema fields	12	13	11	14	0.7857
Field availability at 5% threshold	10	11	9	12	0.7500

Table 10 Repository-specific field availability using the 5% coverage threshold.

Availability Class	Final-dataset Fields
Shared	<code>paper_title</code> , <code>paper_abstract</code> , <code>publication_url</code> , <code>repository_url</code> , <code>repository_title</code> , <code>readme_content</code> , <code>somef_description</code> , <code>tasks</code> , <code>secondary_labels</code>
Papers with Code only	<code>methods</code>
bio.tools only	<code>software_name</code> , <code>software_description</code>
Below threshold in both	<code>repository_description</code> , <code>repository_keywords</code>

7 Research Software Classification Experiments and Results

7.1 Experiment Status

Table 11 records the current status of the classification experiments. Fine-grained metrics are reported only for rows with validated saved outputs.

The verified held-out test split contains 37,058 Papers with Code single-label records, with 29,646 training records and 7,412 test records across six labels. The split is deterministic and stratified with random seed 42.

7.2 Verified Held-out Test Results

Among the validated rows, LinearSVC has the highest macro-F1, while XGBoost has the highest accuracy and micro-F1. The divergence between macro-F1 and accuracy shows why macro-F1 is the primary metric: high accuracy can coincide with weaker rare-label performance.

7.3 Label Support and Class-level Results

Class-level reporting uses a minimum test-support threshold of 10. This threshold affects descriptive class-level tables only; it does not remove classes from aggregate macro-F1 computation. The complete unfiltered classification reports remain in the replication package.

Table 11 Experiment status for the current manuscript revision.

Experiment family	Implemented	Run completed	Results validated	Included here
TF-IDF sparse baselines, PwC single-label abstract-only	Yes	Yes	Yes	Yes
Other sparse baseline datasets and attribute sets	Yes	No	No	No
Frozen transformer embeddings	Yes	No	No	No
Local LLM zero-shot prompting	Interface only	No	No	No
Local LLM one-shot prompting	Interface only	No	No	No
Local LLM few-shot prompting	Interface only	No	No	No

Table 12 Validated held-out test results currently saved in `final_results/tables/classification_results.csv`. All rows use Papers with Code single-label records and paper abstracts only.

Pipeline	Macro-F1	Micro-F1	Weighted-F1	Accuracy
TF-IDF LinearSVC	0.7249	0.8687	0.8691	0.8687
TF-IDF hard voting	0.7143	0.8672	0.8668	0.8672
TF-IDF XGBoost	0.6705	0.8764	0.8722	0.8764
TF-IDF logistic regression	0.6468	0.8176	0.8210	0.8176
TF-IDF random forest	0.6282	0.8620	0.8550	0.8620

Table 13 Label support for the verified Papers with Code single-label split. All classes pass the reporting threshold of 10 test examples.

Label	Train	Test	Share (%)	Reported
Computer Vision	14,128	3,532	47.66	Yes
Natural Language Processing	10,125	2,532	34.15	Yes
Graphs	3,069	767	10.35	Yes
Reinforcement Learning	1,208	302	4.07	Yes
Sequential	1,064	266	3.59	Yes
Audio	52	13	0.18	Yes

Table 14 Per-class F1 extremes for the verified TF-IDF hard-voting row, restricted to classes with at least 10 test examples.

Extreme	Label	Precision	Recall	F1
Lowest	Audio	0.2941	0.3846	0.3333
Highest	Computer Vision	0.8958	0.9077	0.9017

The support distribution is highly imbalanced. Audio has only 13 test records and remains difficult even though it passes the reporting threshold; Computer Vision and Natural Language Processing dominate the test set.

7.4 Removed and Pending Results

Legacy all-metadata TF-IDF LinearSVC rows in older result folders are not reported as findings because the historical `all_available_metadata` predictor definition included leakage-prone fields such as `tasks`, `methods`, `secondary_labels`, and in some scripts `labels`. Those rows must be rerun with the new `all_non_target_textual_metadata` definition before they can support claims. Frozen-embedding and local-LLM experiments are also pending validated outputs.

8 Discussion

8.1 Answers to the Research Questions

RQ1 asked which attributes are represented and how they can be organized. The final retained package supports a unified framework covering identifiers, names, descriptions, publication metadata, research domains, functionality, repository links, and documentation. It does not retain structured input/output data, data formats, programming languages, licenses, benchmarks, or evaluation metrics as final analysis fields.

RQ2 asked how the repositories overlap. Literal label overlap is zero, but schema-field overlap is substantial, with a Jaccard coefficient of 0.7857. Field-availability overlap is also high, with a Jaccard coefficient of 0.7500. These values are evidence of structural comparability, not evidence that the two vocabularies are semantically aligned.

RQ3 asked how the repositories differ in coverage, completeness, richness, and repository-specific fields. Both have high completeness under the revised denominator. `bio.tools` has more labels per record and slightly more populated fields, while Papers with Code has more unique controlled semantic values and greater variability. Dedicated software-name and software-description fields are `bio.tools`-specific at the field level, while methods are Papers with Code-specific.

RQ4 asked what the currently validated classification results show. The current held-out test evidence is limited to Papers with Code single-label classification from abstracts. In that setting, LinearSVC has the highest verified macro-F1 among the

TF-IDF baselines, while XGBoost has the highest accuracy. Macro-F1 remains lower than accuracy because rare labels are difficult.

RQ5 asked which experiments remain pending. Additional sparse baselines across all datasets and leakage-audited attribute sets need to be rerun. Frozen transformer embeddings and local LLM zero-shot, one-shot, and few-shot prompting are implemented or specified but do not yet have validated full outputs.

8.2 Semantic Metadata and Predictive Utility

The semantic analysis and classification results should be interpreted together but not conflated. High field coverage does not by itself guarantee predictive utility, and the current verified classification rows cover only abstract text. Claims about richer non-target metadata, repository text, frozen embeddings, or local prompting require completed leakage-audited runs.

8.3 Implications for Metadata Standards and Repositories

The final retained package lacks structured programming language, license, data-format, benchmark, and evaluation-metric fields. These dimensions are relevant to standards such as CodeMeta, CFF, EDAM, and software registry schemas, but they cannot be analysed unless extraction pipelines preserve them in the final analysis package. Repositories and harmonization workflows should retain structured source-level fields, provenance, and controlled identifiers alongside textual descriptions.

8.4 Implications for Knowledge Graphs and Interoperability

The semantic-overlap results support a knowledge-graph view of interoperability. Direct vocabulary matching is insufficient, but schema-level and concept-level alignment can reveal shared structure. Future integrations should preserve provenance for each mapped field and should support explicit ontology alignment between Papers with Code concepts and EDAM categories. Ontology matching methods provide a possible direction for this future work [25–27].

9 Threats to Validity

9.1 Dataset and Selection Validity

The final datasets include only records retained by the implemented pipeline. Filtering by repository URL excludes software without such links. bio.tools records are affected by publication expansion, DOI-linked publication metadata, and EDAM top-level mapping. The repositories also represent different domains: Papers with Code is concentrated in artificial intelligence, whereas bio.tools covers Life Science software. These domain differences confound direct source-to-source interpretation.

9.2 Construct Validity

Completeness and richness are operational definitions over retained final fields, not universal measures of metadata quality. The 5% availability threshold is a pragmatic

cutoff and may hide rare but meaningful metadata. Literal label overlap underestimates conceptual similarity because no ontology-level alignment between Papers with Code labels and EDAM labels is implemented. Coverage gaps for programming language, license, format, benchmark, and evaluation metadata refer to retained final fields rather than to all possible source-level metadata.

9.3 Internal Validity

Preprocessing choices influence the final representation. Title-based duplicate handling, text normalization, missing-value treatment, and enrichment quality may affect coverage and classification. Repeated repositories or related publications may introduce dependence between records. The verified classification experiments use one deterministic 80/20 split rather than repeated splits or cross-validation. Robustness across alternative splits remains untested for the current validated rows.

9.4 Model-selection and Evaluation Validity

The validated classification rows are preliminary exploratory results from a single held-out split. They should not be treated as final model-selection estimates. Future work should use validation splits or cross-validation for model and hyperparameter selection, with the final test set used only once. Prompting results may be sensitive to prompt wording, local model snapshot, quantization, response parsing, and runtime hardware. Frozen-embedding results may depend on model cache state, package versions, device, and pooling implementation.

9.5 External Validity and Reproducibility

The study covers two repositories and should not be generalized without caution to other Research Software registries, package indexes, institutional catalogs, or archival infrastructures. The analyses are grounded in repository scripts and generated output files, and random seed 42 is recorded for implemented train/test experiments. The current manuscript excludes unsupported and leakage-prone historical results; final reproducibility for pending model families depends on rerunning and validating those outputs under the updated protocol.

10 Conclusion

This paper compared semantic metadata attributes from Papers with Code and bio.tools and defined a leakage-audited classification protocol for Research Software metadata. The final harmonized dataset contains 56,819 labelled records, with 42,669 from Papers with Code and 14,150 from bio.tools. The semantic analysis shows no literal overlap between final label vocabularies, but substantial overlap at schema-field and field-availability levels. Metadata completeness is high in both repositories when computed over represented applicable fields, while semantic richness differs across sources.

The completed, validated classification evidence is narrower than the original experimental plan: it currently covers TF-IDF sparse baselines for Papers with Code

single-label classification using abstracts only. In that verified setting, LinearSVC reaches macro-F1 0.7249 and accuracy 0.8687 on the held-out test set. Additional sparse baselines over all leakage-audited attribute sets, frozen transformer embedding experiments, local LLM zero-shot/one-shot/few-shot experiments, stronger evaluation, and imbalance-aware analysis remain in progress.

Future work should rerun all affected experiments with `all_non_target_textual_metadata`, add ontology alignment between Papers with Code and EDAM, evaluate additional Research Software repositories, preserve currently unrepresented structured fields, study cross-domain transfer, and improve multi-label classification.

Supplementary information

The replication package contains the final datasets, generated semantic tables, classification outputs, figures, and scripts used for the analyses.

Acknowledgements

Declarations

Funding

Competing interests

Data availability

Code availability

References

- [1] Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Silva Santos, L.B., Bourne, P.E., *et al.*: The fair guiding principles for scientific data management and stewardship. *Scientific Data* **3**, 160018 (2016) <https://doi.org/10.1038/sdata.2016.18>
- [2] Lamprecht, A.-L., Garcia, L., Kuzak, M., Martinez, C., Arcila, R., Martin Del Pico, E., Dominguez Del Angel, V., Sandt, S., Ison, J., Martinez, P.A., McQuilton, P., Valencia, A., Harrow, J., Psomopoulos, F., Gelpi, J.L., Chue Hong, N., Goble, C., Capella-Gutierrez, S.: Towards fair principles for research software. *Data Science* **3**(1), 37–59 (2020) <https://doi.org/10.3233/DS-190026>
- [3] Barker, M., Chue Hong, N.P., Katz, D.S., Lamprecht, A.-L., Martinez-Ortiz, C., Psomopoulos, F., Harrow, J., Castro, L.J., Gruenpeter, M., Martinez, P.A., Honeyman, T., *et al.*: Introducing the fair principles for research software. *Scientific Data* **9**, 622 (2022) <https://doi.org/10.1038/s41597-022-01710-x>

- [4] FAIR for Research Software Working Group: FAIR Principles for Research Software (FAIR4RS Principles). Zenodo (2022). <https://doi.org/10.15497/RDA00068> . <https://doi.org/10.15497/RDA00068>
- [5] Smith, A.M., Katz, D.S., Niemeyer, K.E., Group, F.S.C.W.: Software citation principles. *PeerJ Computer Science* **2**, 86 (2016) <https://doi.org/10.7717/peerj-cs.86>
- [6] Papers with Code: Papers with Code. <https://paperswithcode.com/>. Accessed 2026-07-13 (2026)
- [7] Ison, J., Kalaš, M., Jonassen, I., *et al.*: Edam: an ontology of bioinformatics operations, types of data and identifiers, topics and formats. *Journal of Biomedical Semantics* **4**, 43 (2013) <https://doi.org/10.1186/2041-1480-4-43>
- [8] Ison, J., Ienasescu, H., Chmura, P., Raschi, E., Gavras, H., Blankenberg, D., Jimenez, R.C., *et al.*: The bio.tools registry of software tools and data resources for the life sciences. *Genome Biology* **20**(1), 164 (2019) <https://doi.org/10.1186/s13059-019-1772-6>
- [9] Ison, J., Ienasescu, H., Chmura, P., *et al.*: biotoolsschema: a formalized schema for bioinformatics software metadata. *Database* **2021** (2021) <https://doi.org/10.1093/database/baaa157>
- [10] Katz, D.S.: Transitive credit as a means to address social and technological concerns stemming from citation and attribution of digital products. *Journal of Open Research Software* **2**(1), 20 (2014) <https://doi.org/10.5334/jors.be>
- [11] Manghi, P., Atzori, C., Bardi, A., *et al.*: The openaire research graph: A fair scholarly knowledge graph. *Scientific Data* **10**, 141 (2023) <https://doi.org/10.1038/s41597-023-01971-9>
- [12] Di Cosmo, R.: Archiving and referencing source code with software heritage. In: *International Conference on Mathematical Software (ICMS). Lecture Notes in Computer Science*, vol. 12097, pp. 362–373. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-52200-1_36
- [13] Jones, M.B., Boettiger, C., Mayes, A.C., Smith, A.M., Slaughter, P., Niemeyer, K.E., Gil, Y., Fenner, M., Nowak, K., Hahnel, M., Coy, L., Allen, A., Crosas, M., Sands, A., Clune, J., Jackson, M., Michel, F., Goble, C.: Codemeta: An exchange schema for software metadata. *Journal of Open Research Software* **7**(1), 37 (2019) <https://doi.org/10.5334/jors.247>
- [14] Druskat, S., Spaaks, J., Hong, N.C., *et al.*: Citation File Format 1.2.0. Specification (2021). <https://citation-file-format.github.io/>
- [15] Malone, J., Brown, A., Lister, A., *et al.*: The software ontology (swo): a resource

for reproducibility in biomedical data analysis, curation and digital preservation. *Journal of Biomedical Semantics* **5**, 25 (2014) <https://doi.org/10.1186/2041-1480-5-25>

- [16] Soll, M., Vosgerau, M.: Classifyhub: An algorithm to classify github repositories. In: *KI 2017: Advances in Artificial Intelligence. Lecture Notes in Computer Science*, vol. 10505, pp. 373–379. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-67190-1_34
- [17] Zhang, Y., Xu, F.F., Li, S., Meng, Y., Wang, X., Li, Q., Han, J.: Higitch: Keyword-driven hierarchical classification of github repositories. In: *2019 IEEE International Conference on Data Mining (ICDM)*, Piscataway, NJ, USA, pp. 876–885 (2019). <https://doi.org/10.1109/ICDM.2019.00154> . <https://doi.org/10.1109/ICDM.2019.00154>
- [18] Sas, C., Capiluppi, A.: Labelgit: A dataset for software repositories classification using attributed dependency graphs. *arXiv preprint arXiv:2103.08890* (2021)
- [19] Prana, G.A.A., Treude, C., Thung, F., Atapattu, T., Lo, D.: Categorizing the content of github readme files. *Empirical Software Engineering* **24**, 1296–1327 (2019) <https://doi.org/10.1007/s10664-019-09691-3>
- [20] Tsay, J., Braz, A., Hirzel, M., Shinnar, A., Mummert, T.: Aimmx: Artificial intelligence model metadata extractor. In: *Proceedings of the 17th International Conference on Mining Software Repositories*, pp. 81–92. ACM, New York, NY, USA (2020). <https://doi.org/10.1145/3379597.3387448> . <https://doi.org/10.1145/3379597.3387448>
- [21] Garijo, D., Osorio, M., Khider, D., Ratnakar, V., Gil, Y.: Okg-soft: An open knowledge graph with machine readable scientific software metadata. In: *2019 IEEE 15th International Conference on eScience*, pp. 172–181. IEEE, Piscataway, NJ, USA (2019). <https://doi.org/10.1109/eScience.2019.00027> . <https://doi.org/10.1109/eScience.2019.00027>
- [22] Mao, A., Garijo, D., Fakhraei, S.: Somef: A framework for capturing scientific software metadata from its documentation. In: *2019 IEEE International Conference on Big Data (Big Data)*, pp. 3032–3037. IEEE, Piscataway, NJ, USA (2019). <https://doi.org/10.1109/BigData47090.2019.9006447> . <https://doi.org/10.1109/BigData47090.2019.9006447>
- [23] Kelley, A., Garijo, D.: A framework for creating knowledge graphs of scientific software metadata. *Quantitative Science Studies* **2**(4), 1423–1446 (2021) <https://doi.org/10.1162/qss.a.00167>
- [24] Salatino, A.A., Osborne, F., Thanapalasingam, T., Motta, E.: The cso classifier: Ontology-driven detection of research topics in scholarly articles. In: *Theory and Practice of Digital Libraries*, pp. 296–311. Springer, Cham (2019). https://doi.org/10.1007/978-3-319-98890-1_18

[org/10.1007/978-3-030-30760-8_20](https://doi.org/10.1007/978-3-030-30760-8_20)

- [25] Euzenat, J., Shvaiko, P.: *Ontology Matching*, 2nd edn. Springer, Berlin, Heidelberg (2013). <https://doi.org/10.1007/978-3-642-38721-0> .
<https://doi.org/10.1007/978-3-642-38721-0>
- [26] Otero-Cerdeira, L., Rodríguez-Martínez, F.J., Gómez-Rodríguez, A.: *Ontology matching: A literature review*. *Expert Systems with Applications* **42**(2), 949–971 (2015) <https://doi.org/10.1016/j.eswa.2014.09.003>
- [27] Trojahn, C., *et al.*: *Foundational ontologies meet ontology matching: A survey*. *Semantic Web* **13**(3), 449–479 (2022) <https://doi.org/10.3233/SW-210447>